



DESARROLLO DE ALGORITMOS DE PERCEPCIÓN Y CONTROL EN ROS2 PARA ROBÓTICA MÓVIL



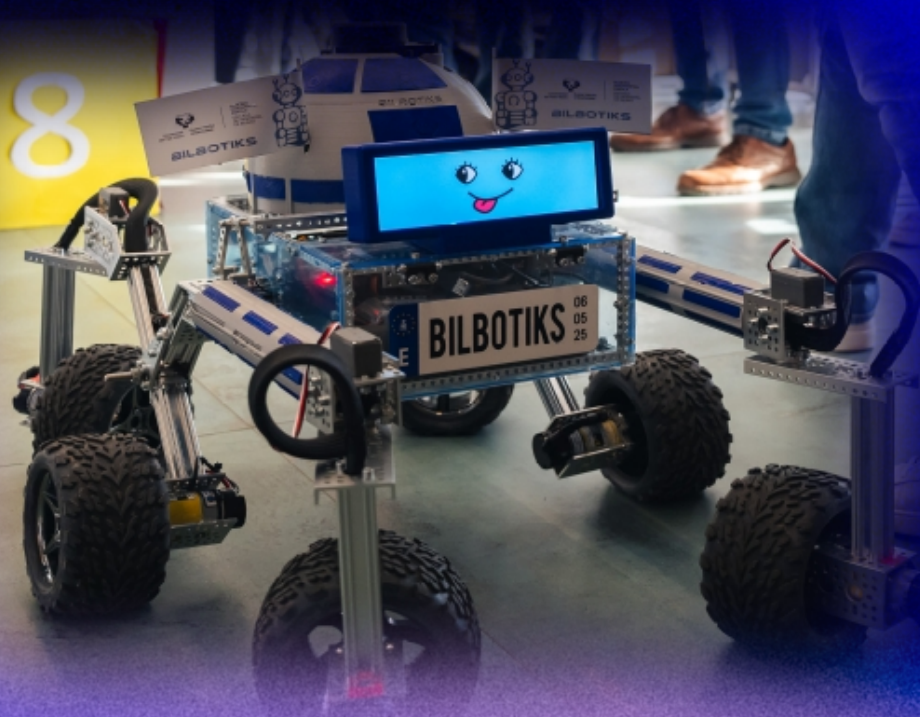
Javier Arambarri Calvo

Ingeniería Informática de Gestión y
Sistemas de Información



Índice

1. Sobre mí
2. Introducción y objetivo
3. Pruebas a resolver
4. Diseño y montaje del rover
5. Diseño de la arquitectura
6. Algoritmos de percepción y control
7. Resultados
8. Familia BilboTiks



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



1. Sobre mí



→ Graduado en Ingeniería Informática de Gestión y Sistemas de Información – Escuela de Ingeniería de Bilbao – UPV/EHU

→ Estudiante de Máster en Ingeniería de Sistemas y Control – UCM y UNED

→ Investigador contratado en la Escuela de Ingeniería de Bilbao.

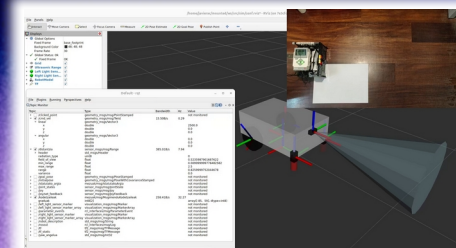


Proyectos personales:

→ EXPLORANDO LA ROBÓTICA: Guía para First Lego League y World Robot Olympiad.
Libro disponible en Amazon.

→ eduROS2: robótica educativa Lego con ROS2.

→ ... ¡y más!



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



2. Introducción y objetivo

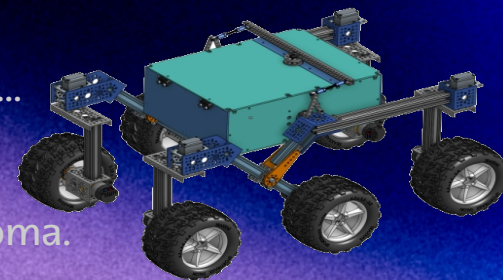
"Desarrollar algoritmos de percepción en el entorno y control que permitan a un rover identificar números y colores y desplazarse por un entorno cerrado."

Robótica móvil: diseño y desarrollo de robots autónomos en entornos desconocidos para asistir a personas en la realización de tareas: logística, agricultura, salud, exploración submarina y espacial...

AUGE IMPORTANCIA DEL SOFTWARE: se necesitan sistemas de control más sofisticados e inteligentes → mayor autonomía.

Tendencias del sector de la robótica 2025 según IFR: **IA, robótica móvil**, ...

Sener-CEA's Bot Talent: concurso universitario para diseñar robots móviles con capacidades de percepción, control, guiado y navegación autónoma.



JPL Open Source Rover Project

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



3. Pruebas a resolver

Percepción:

Realizar un giro sobre sí mismo en el sentido indicado por el color de la caja y tantas vueltas como el dígito de la caja.

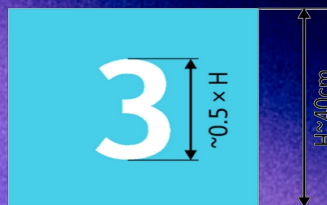
El color podrá ser:

→ azul: sentido antihorario,

→ rojo: sentido horario,

→ cualquier otro que se cataloga como "distractor"

Los números podrán ser del 1 al 9, incluyendo la categoría de "sin número".

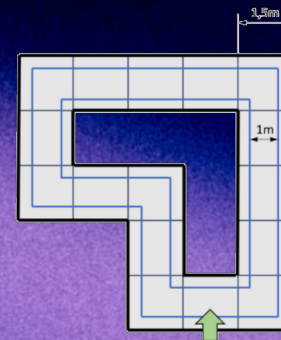


Control:

Recorrer un laberinto cerrado, con anchura 1,5 m y giros de 90 grados manteniéndose lo más centrado posible en el carril y sin chocarse con las paredes.

Para evaluar lo centrado que se desplaza el robot, se utilizan Líneas de Control (LdC) de 1 m de ancho.

Se establece un máximo de 5 minutos para completarla.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



4. Diseño y montaje del rover

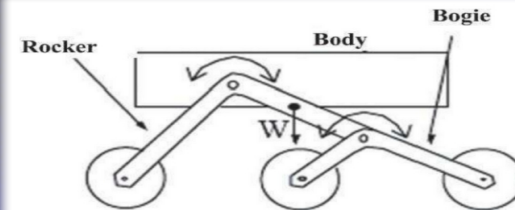
El concurso proporciona la plataforma **JPL Open Source Rover Project**: versión simplificada y open source del robot espacial (rover) de **6 ruedas** Perseverance de la NASA.

Suspensión rocker-bogie: el rocker se une a la caja mediante una articulación diferencial, cuando un lado sube, el otro baja. El bogie se une al rocker mediante una articulación libre. Permite superar obstáculos un 50% mayores que el diámetros de las ruedas.

Componentes principales:

Motores con encoders y controladores Roboclaw

Servomotores direccionamiento



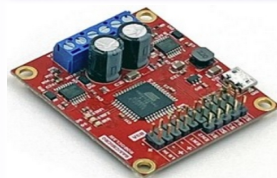
Placa base



Cámara y sensor de profundidad.



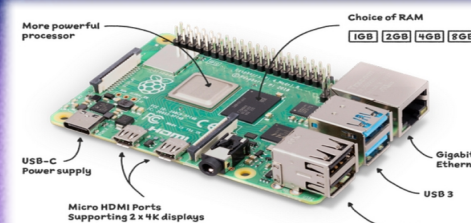
LiDAR 2D



IMU



Raspberry Pi 4B de 8GB RAM



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

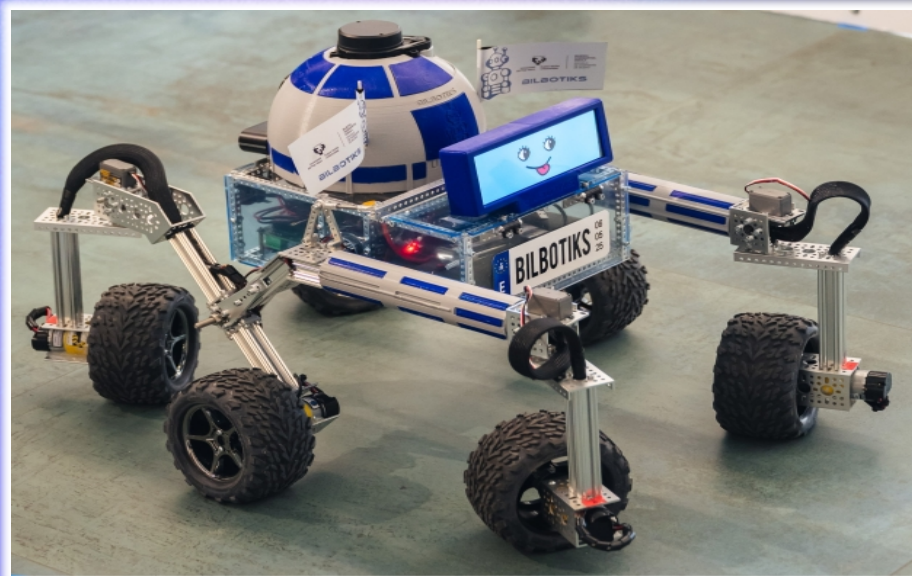
Javier Arambarri Calvo



4. Diseño y montaje del rover

Montaje

Cúpula inspirada en R2-D2 de Star Wars y marco impresos en 3D para albergar la cámara, IMU, LiDAR y pantalla.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



5. Diseño de la arquitectura

3 niveles de abstracción:

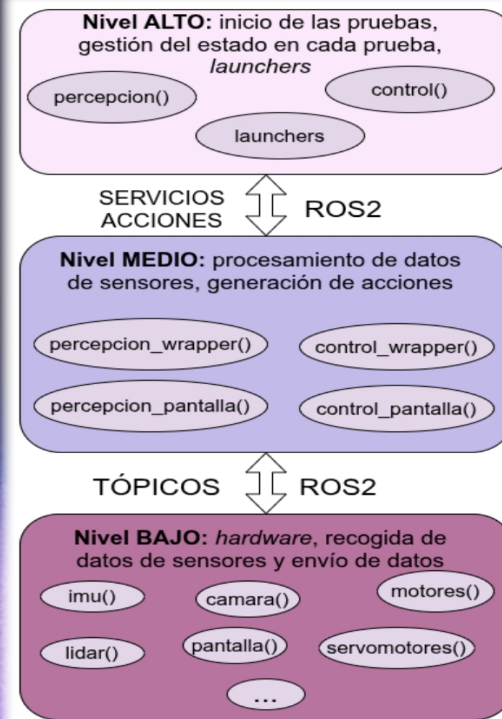
→ Permite separar la gestión y control del hardware de la lógica de aplicación

MUY IMPORTANTE porque permite dividir una gran tarea en diferentes subtareas o submódulos y reutilizar código:

→ Módulo o nodo de la IMU: gestiona la IMU.

→ Módulo de la prueba de percepción: utiliza el módulo de la IMU para evitar repetir el código de gestión del sensor.

Facilidad de depuración y aumenta la modularidad.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos

Universidad
del País VascoEuskal Herriko
UnibertsitateaBILBAO
SCHOOL
OF ENGINEERING
UNIVERSITY
OF THE BASQUE
COUNTRY

ROS2 – Robot Operating System: framework de código abierto para el desarrollo de aplicaciones de robótica móvil en Python y C++.

- **Nodo:** ejecutable que desempeña una tarea. Son parametrizables. Se comunican mediante:
 - **Tópicos:** publicador-suscriptor asíncrono. Información de sensores en tiempo real.
 - **Servicios:** cliente-servidor síncrono. Operaciones puntuales de respuesta inmediata.
 - **Acciones:** asíncrona basada en tópicos y servicios. Tareas largas con retroalimentación.

NODOS LIFECYCLE:

Nodos o ejecutables con ciclo de vida → **FSM o Máquina de Estados**. Permite activar/desactivar nodos. Muy útil para controlar el estado del dispositivo físico.

Por ejemplo: en la prueba de percepción la cámara solo se activa en el momento de la foto. El nodo está disponible, pero inactivo evitando consumir grandes recursos de computación.



6. Algoritmos

GRAFO ROS2

tres paquetes ROS2:

→ **bilbotiks_controller**

→ **bilbotiks_interfazeak**:
mensajes

→ **bilbotiks_pantaila**: GUI

Nodos L: Lifecycle

Nodos: nivel alto

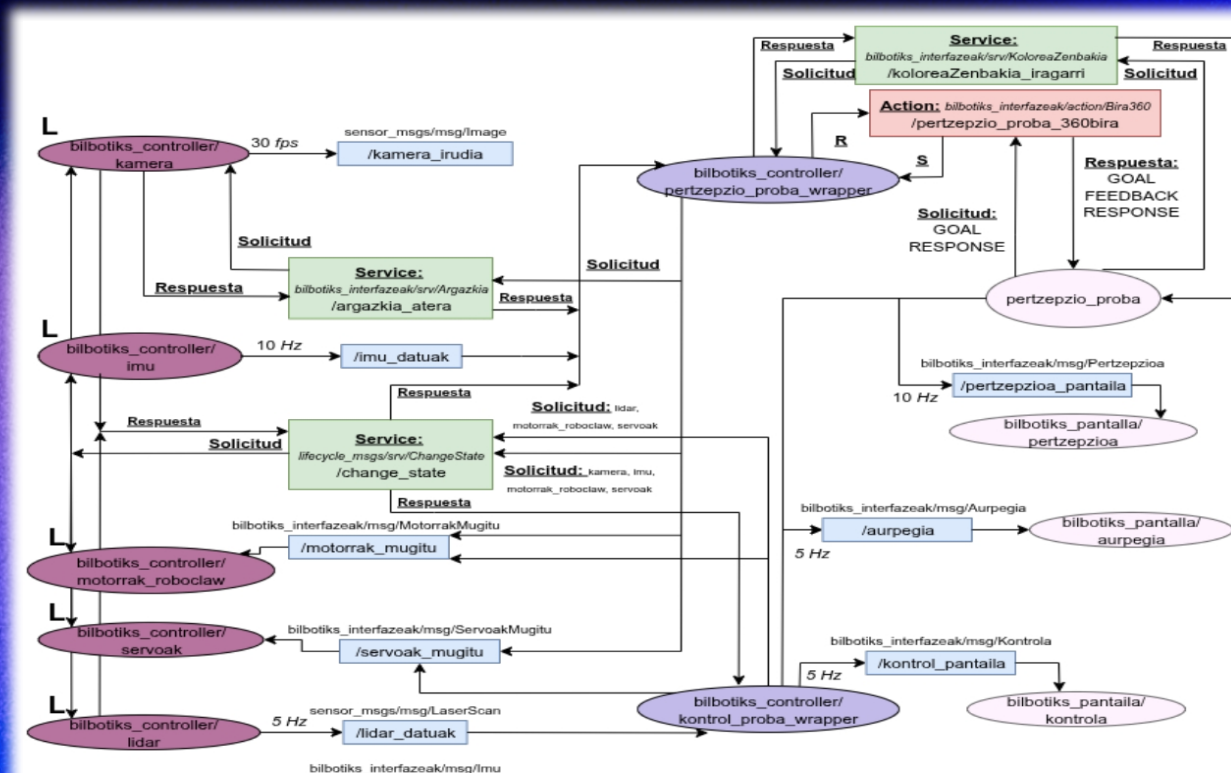
Nodos: nivel medio

Nodos: nivel bajo

Tópicos

Servicios

Acciones



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de percepción

1. Identificar y recortar la sección coloreada.
2. Procesar el recorte en busca del número.

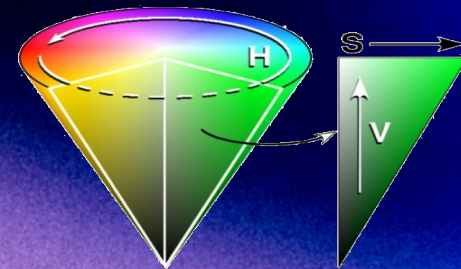


Identificación del color:

Codificación digital de la imagen y su espacio de color HSV.

Representa los colores de manera similar al ser humano en 3 componentes: tono, saturación y valor o luminosidad.

Más sencillo que RGB para obtener las diferentes variantes de cada color. En RGB, ¿cómo sabemos qué cantidad de cada canal componen un color con determinado brillo?



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



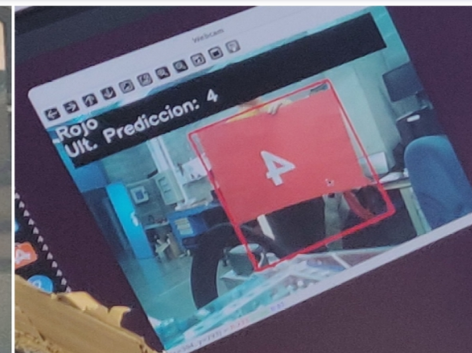
6. Algoritmos de percepción

Identificar la caja: restricciones geométricas (visión clásica)

Evitan identificar píxeles sueltos como la caja.

→ Recorte + Zoom: recorte inferior del 30%, lateral 20%, zoom centrado del 57%.

→ Rango de tamaño de píxeles que puede ocupar la caja, considerando la distancia y su tamaño.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de percepción

Identificar el número

→ Generación de un **DATASET** específico con **10.736 imágenes**.

Incluye imágenes sintéticas para reforzar aprendizaje.
Por último, cribado manual para retirar aquellas que introduzcan confusión y no aporten valor.

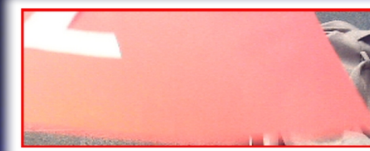


Tres subconjuntos (100%):

- Entrenamiento: 70%, usado para aprender.
- Validación: 15%, ajustar hiperparámetros.
- Desarrollo: 15%, medir el rendimiento final.



Ejemplo de imágenes sintéticas



Ejemplo de imágenes eliminadas

→ Se aplica el **algoritmo de bordes "Canny"** al conjunto de datos completo para obtener los bordes.

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de percepción

Identificar el número

- Red neuronal convolucional (**CNN**).
- 3 capas convolucionales.
- 1 capa de aplanado.
- 2 capas densas:
- Capa de salida: 10 neuronas.

Técnicas utilizadas para prevenir overfit:

- Normalización de batch
- Dropout del 30%

Tamaño de lote: 64

Learning rate dinámico: empieza en 0,001

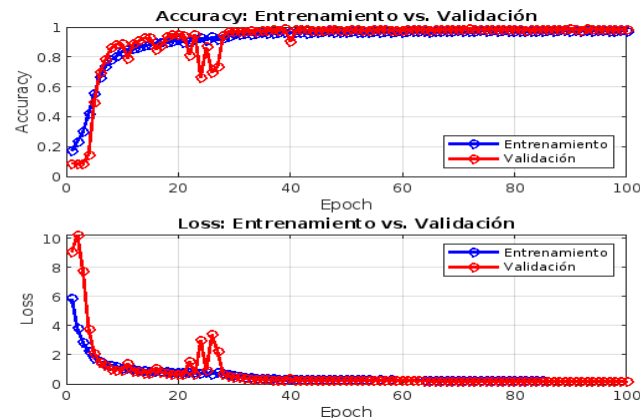
Función de pérdida: categorical crossentropy

Optimizador Adam

Evaluación - Resultados

→ Validation accuracy: proporción de aciertos sobre el dataset de validación.

→ Validation loss: cuánto de lejos están las predicciones del modelo de los valores reales. Se busca que sea cercano a 0.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



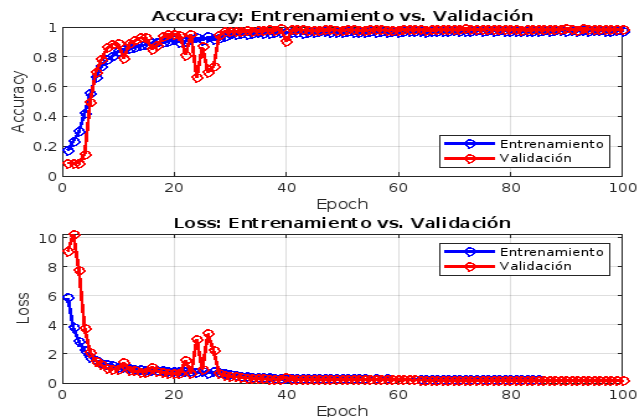
6. Algoritmos de percepción

Universidad
del País VascoEuskal Herriko
UnibertsitateaBILBAO
SCHOOL
OF ENGINEERING
UNIVERSITY
OF THE BASQUE
COUNTRY

Evaluación - Resultados

→ Validation accuracy: proporción de aciertos sobre el dataset de validación.

→ Validation loss: cuánto de lejos están las predicciones del modelo de los valores reales. Se busca que sea cercano a 0.



Sobreajuste si:

1. validation accuracy es significativamente menor que la training accuracy
2. training loss disminuye continuamente, pero la validation loss se estabiliza o incluso aumenta

Training accuracy (máx 1)	0.9773
Training loss (máx ∞)	0.1976
Validation accuracy (máx 1)	0.9896
Validation loss (máx ∞)	0.1514

Modelo resultante: fichero de 8,4 MB.

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de percepción

Rendimiento – TIP PARA EL HACKATHON !!

Uso de 6GB de RAM para la inferencia!! Solo para detectar un dígito!!

No dió tiempo y era aceptable... pero para solucionarlo:

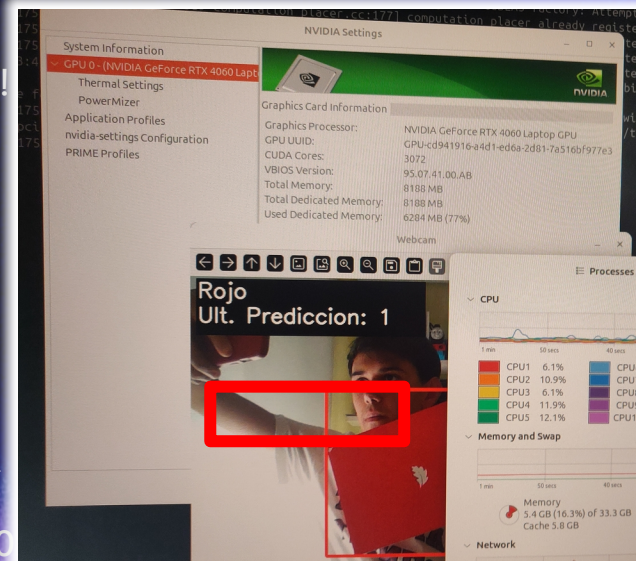
→ Reducir la dimensionalidad de las imágenes: al aplicar Canny se trabaja con imágenes binarias, por lo que pueden reducirse a unas pocas decenas de píxeles.

→ Capas convolucionales demasiado profundas.

→ Usa GlobalAveragePooling2D en lugar de Flatten (capa aplanado)

Si la última capa convolucional produce, por ejemplo, un mapa de características de tamaño 50·50·128, con el aplanado se obtienen 320 64: $64 \cdot 320.000 = 20,4$ millones de valores que pasan a la capa densa.

GlobalAveragePooling2D Reduce cada mapa de características a un solo valor promedio → pasas de cientos de miles de entradas a solo unas decenas: si la última convolucional tiene 128 filtros, tras GlobalAveragePooling2D obtienes solo 128 valores.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de percepción



TIP PARA EL HACKATHON !!

Muchos casos de uso pueden ser solucionados con técnicas clásicas de visión de computador:

- Obtención de bordes/regiones mediante la transformada de Hough
- Obtener siluetas de objetos mediante contornos deformables
- Vaciar objetos mediante operaciones morfológicas
- Descomponer objetos en cantidad de rectas direccionadas que describen los objetos (algoritmo de Tanaka)
- Detección de bordes Canny, suavizado gaussiano...
- ...

Suelen tener una aplicación directa mediante librerías como OpenCV, por lo que simplifican la complejidad del problema y ahorran tiempo.

Repositorio propio con la memoria y código de las tareas desarrolladas en la asignatura de Visión por Computador del Máster en Ingeniería de Sistemas y Control de la UCM-UNED (accesible desde mi Linktree, QR diap. 3):

https://github.com/arambarricalvoj/vpc_misc_ucm-uned_25-26

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de percepción

Giros con IMU: precisión y sencillez

Se ha desarrollado un algoritmo de doble etapa que permite disminuir artificialmente la latencia del sensor cuando el robot está próximo a su objetivo con el fin de aumentar la precisión del giro:

→ Primera etapa: mientras no se haya superado el 80 % de los grados a girar, es decir, $0'8 \cdot 360^\circ \cdot \text{número de vueltas a girar}$:

- Leer IMU y mover el rover a velocidad RÁPIDA.
- Si se ha dado una vuelta, contarla.
- Actualizar grados girados.

→ Segunda etapa: mientras no se haya superado el 100 % de los grados a girar, es decir, $360^\circ \cdot \text{número de vueltas a girar}$:

- Leer IMU y mover el rover a velocidad LENTA.
- Si se ha dado una vuelta, contarla.
- Actualizar grados girados.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de control

1. Obtener la distancia a las paredes del entorno.
2. Generar una respuesta en la dirección del movimiento.

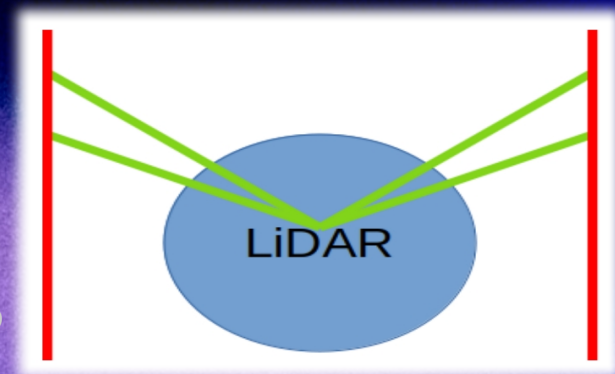
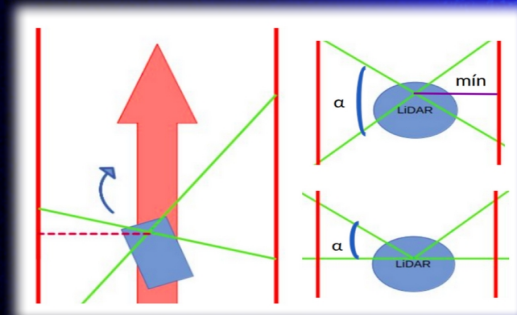
Estrategia:

error de posición = distancia a la pared izquierda – distancia a la pared derecha

Siendo el laberinto de 1,5 metros de ancho, ese será el error máximo (simplificado, no se tiene en cuenta la anchura del robot).

Campo de visión implementado: 22,5° cada lateral → obtiene la pared "futura", anticipa el giro.

Dentro de este rango de valores posibles, se escoge el mínimo como distancia lateral.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de control

Universidad
del País VascoEuskal Herriko
UnibertsitateaBILBAO
SCHOOL
OF ENGINEERING
UNIVERSITY
OF THE BASQUE
COUNTRY

Interpolación lineal para mover los servomotores: conociendo las posiciones límite de cada servomotor y que el ángulo de giro están comprendido en el rango $[-90, 90]$.

$$\text{ángulo servomotor} = \text{máx derecho} - \left(\frac{\text{ángulo de giro}}{90} \right) \cdot (\text{máx derecho} - \text{mín derecho})$$

Control PD: sistema de control para generar una señal de salida de corrección relacionada con la señal de entrada (error).

Sintonización de constantes mediante Ziegler-Nichols en lazo cerrado y **ensayo y error***:

→ K_p : 100

→ K_d : 25

→ Velocidad: 30 unidades sobre 127 posibles en la librería del fabricante de los motores.

***ANÁLISIS:** ¿valor mínimo para K_p ? Corrección máxima es de 90° cuando el error es máximo:

$$K_{p_{\min}} = \frac{\text{Corrección máxima}}{\text{Error máximo}}$$

$$K_{p_{\min}} = \frac{90}{1,5} = 60$$

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos de control

Evaluación – Resultados del control PD



(a) $K_p = 120$, $K_d = 45$,
velocidad = 30, campo visión = 90°



(b) $K_p = 120$, $K_d = 45$,
velocidad = 30, campo visión = 22.5°



(c) $K_p = 100$, $K_d = 25$,
velocidad = 30, campo visión = 90°



(d) $K_p = 100$, $K_d = 25$,
velocidad = 30, campo visión = 22.5°

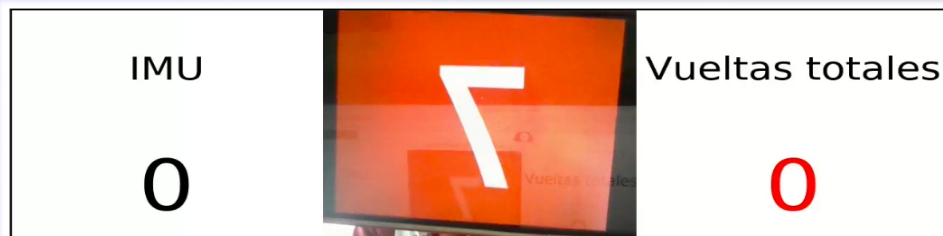
Relación K_p - K_d : mayor peso K_p
→ K_p alto: mayor rapidez / brusquedad
→ Mayor sobreimpulso
→ K_d no logra amortiguar lo suficiente

Relación K_p - K_d : más equilibrada
→ K_p baja: menor rapidez / brusquedad
→ K_d influye más: mayor amortiguamiento



6. Algoritmos

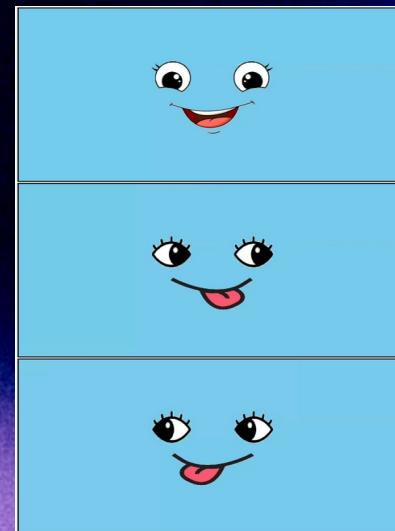
Interfaces gráficas: una pantalla visual por cada prueba para monitorizar el proceso y una cara amigable dinámica para “humanizar” el robot. Controladas mediante tópicos de ROS2.



Pantalla de la prueba de percepción



Pantalla de la prueba de control



Cara amigable dinámica

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Algoritmos

Gemelo funcional y simulación: componente software para probar el correcto funcionamiento de un sistema de control o algoritmo, respondiendo mediante activaciones ficticias o simuladas de las señales de control y valores de sensores.

IMU: comprobar el algoritmo del giro de la prueba de percepción sin necesidad de utilizar el robot.

Simulación: entorno virtual simple para simular la estrategia de la prueba de control.

Se ha utilizado el simulador **Gazebo Harmonic** de ROS2 Jazzy Jalisco sobre la herramienta de virtualización por contenedores Docker.

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

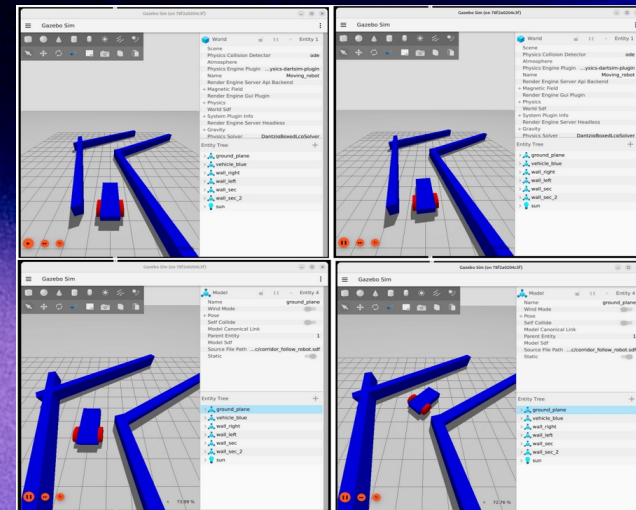
```
javierac@javierac-Standard-PC-Q35-ICH9-2009: ~/Desktop/bilbotiks_ws
$ ros2 run bilbotiks_controller imu_sim
[INFO] [1751979886.595090902] [imu_argitaratzalea]: imu_argitaratzalea nodoa IN constructor
[INFO] [1751979114.514814965] [imu_argitaratzalea]: imu_argitaratzalea nodoa IN on_configure
[INFO] [1751979160.167394494] [imu_argitaratzalea]: imu_argitaratzalea nodoa IN on_activate
[INFO] [1751979160.167805531] [imu_argitaratzalea]: Publicando datos de la IMU en /imu_datauk cada 100ms

giroskopioa:
x: 158.06124664590368
y: 124.53953983924589
z: -484.2923996462349

euler_angeluak:
x: 8.902705700848603
y: 288.1392373108696
z: 240.25487953728077

kuaternioak:
x: 0.794626639486246
y: 0.5204209549948813
z: 0.0005674820247857966
w: 0.3126183158184481

azelerazio lineala:
x: 3.415256864788976
y: 1.7779580335995355
z: 9.40778684690935
```



Javier Arambarri Calvo



Bilbao

23



6. Algoritmos

TIP PARA EL HACKATHON !!

***ANÁLISIS: ¿valor mínimo para Kp? Corrección máxima es de 90° cuando el error es máximo:**

$$Kp_{min} = \frac{\text{Corrección máxima}}{\text{Error máximo}}$$

$$Kp_{min} = \frac{90}{1,5} = 60$$

Realizar un análisis de la situación nos ahorra tiempo.

En este caso, teníamos un Kp mínimo del que partir, por lo que con el método de ensayo y error no íbamos a ciegas.

Simulaciones o gemelos funcionales: útiles para una primera evaluación precoz de un primer prototipo
→ Permite identificar estrategias erróneas sin perder tiempo desplegándolas

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Pruebas de algoritmos

Universidad
del País VascoEuskal Herriko
UnibertsitateaBILBAO
SCHOOL
OF ENGINEERING
UNIVERSITY
OF THE BASQUE
COUNTRY

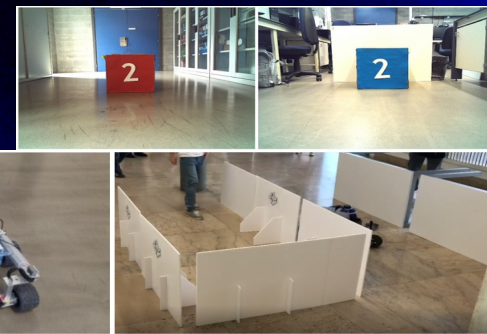
Por cada una de las pruebas del concurso...

- Tests unitarios: validar el funcionamiento individual de cada uno de los procedimientos.
- Tests de integración: verificar el correcto funcionamiento de los nodos de nivel medio y alto, ya que estos invocan a todos los procedimientos verificados mediante las pruebas unitarias.
- Tests de verificación: ¿el sistema ha sido construido correctamente? ¿bien codificado?
- Tests de validación: ¿el sistema correcto ha sido construido? ¿responde a los retos?

Errores solucionados gracias a los test:

- Frecuencia de publicación de las lecturas de la IMU.
- Orientación correcta de las ruedas en los giros.
- Lectura de datos desordenados del LiDAR.
- ...

Las pruebas del concurso se han probado en entornos diferentes con condiciones ambientales distintas, trazados con una anchura inferior y comenzando en una posición con el máximo error posible → Permite identificar dónde falla.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



6. Pruebas de algoritmos

Pruebas - TIP PARA EL HACKATHON !!

→ Separar la lógica de callbacks de ROS2 de la lógica del algoritmo → Permite depurar con facilidad la lógica de la aplicación independientemente de ROS2:

Si la lógica se verifica correcta y al integrarla en ROS2 falla → fallo de uso de ROS2 y no del algoritmo!!!

→ Aprendido en el workshop de ROS2Testing de la ROSConEs25 los días 4-5 noviembre en Barcelona...
... por tanto, el TFG no está desarrollado con esta metodología.

→ Ejemplo en pseudocódigo:

sin separar:

```
function callback(){  
  error = lecturaSensor;  
  corrección = Error * self.kp;  
  publish(corrección);}
```

separando: nodo ROS2

```
importar controlador  
  
function callback(){  
  error = lecturaSensor;  
  corrección = controlador(error);  
  publish(corrección); }
```

separando: controlador

```
function controlador(){  
  corrección = Error * self.kp;  
  return corrección; }
```

Repositorio del workshop con toda la documentación.

https://github.com/Ekumen-OS/ros2_testing_workshop_roscon_es_25

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



7. Resultados

Prueba de percepción

2,5 segundos en percibir el color y el número.
5 segundos por vuelta. El tramo final 4 segundos extra.
Nueve vueltas: $2,5 + 5 \cdot 9 + 4 = \underline{51,5 \text{ segundos.}}$

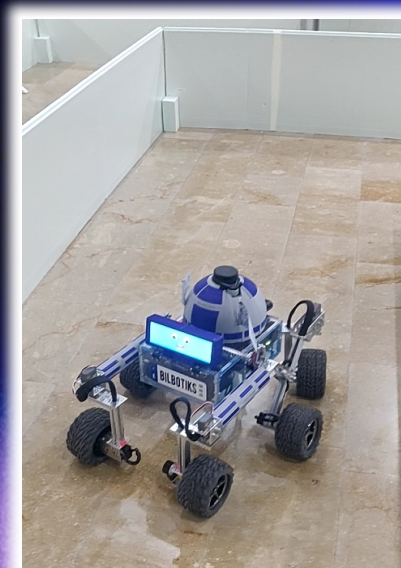
Prueba de control

1 minuto y 5 segundos en dar una vuelta a un circuito sobre una superficie de 50 m².

Controlador PD con parámetros ajustables externamente sin necesidad de recompilar:

- constantes de control,
- velocidad,
- campo de visión del LiDAR.

Solución adaptable a escenarios con diferentes características.



Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

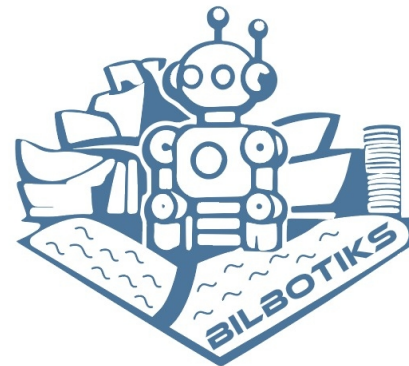
Javier Arambarri Calvo



7. Resultados



1º Premio al Diseño Tecnológico en el Concurso SENER-CEA's Bot Talent

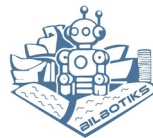


Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



8. Familia BilboTiks



Tutores BilboTiks: Itziar Cabanes Axpe, César Pérez Bravo

Tutores TFG: Itziar Cabanes Axpe, Unai López Novoa

Tutores Sener: Lucía Ballesteros, Lorenzo Prat Boubeta

Estudiantes: Ane Porto Iglesias, Mario Hermo Gordillo, Pablo Elustondo Acuriola, Nagore Toja Arregui, Álvaro Gorczyca Estefanía, Javier Arambarri Calvo

Desarrollo de algoritmos de percepción y control en ROS2 para robótica móvil

Javier Arambarri Calvo



DESARROLLO DE ALGORITMOS DE PERCEPCIÓN Y CONTROL EN ROS2 PARA ROBÓTICA MÓVIL



Javier Arambarri Calvo

Ingeniería Informática de Gestión y
Sistemas de Información